

# Exam - Object-Oriented Programming

Emily Antosch  
HAW Hamburg · Faculty TI

**Winter semester 2026/27 · 90 minutes · 105 points**

Theory: 60 minutes (Tasks 1–5, 65 points). Programming: 30 minutes (Task 6, 40 points).

|       |             |
|-------|-------------|
| Name: | Student ID: |
| Date: | Signature:  |

**Instructions**

- Work individually. Answer Tasks 1–5 in the spaces provided; label any additional sheets with your student ID and task number.
- For Task 6, submit the Java source files in a folder named StudentID\_LastName. Use a standard JDK (Java 17 or later), without preview features or external libraries.
- No GUI, interactive menu, file storage, or concurrency is required. Each task is independent.
- The invigilator will announce permitted aids and the submission channel before the exam starts.
- Check that your paper is complete using the page count in the footer.

**Tasks**

|                                   |   |
|-----------------------------------|---|
| Task 1: Basic Concepts            | 2 |
| Task 2: Object-Oriented Reasoning | 3 |
| Task 3: True or False             | 4 |
| Task 4: Code Completion           | 5 |
| Task 5: Code Debugging            | 6 |
| Task 6: Room Sensor Log           | 7 |

| Task      | 1  | 2  | 3  | 4  | 5  | 6  | Total |
|-----------|----|----|----|----|----|----|-------|
| Available | 10 | 15 | 16 | 12 | 12 | 40 | 105   |
| Achieved  |    |    |    |    |    |    |       |

THEORY · 60 MINUTES IN TOTAL

# Task 1: Basic Concepts

10 points

For each situation, explain the relevant Java concept and give one short Java example. Each item earns **1 point for the explanation and 1 point for the example**.

**a) Object references (2 points).** Two variables should refer to the same Lamp object rather than two separate lamps. Explain what reference assignment copies.

**b) Encapsulation (2 points).** A thermostat must reject target temperatures outside its allowed range. Explain why its temperature field should not be public.

**c) Constructors (2 points).** A Room must receive its name when it is created. Explain the constructor's role and show the constructor declaration.

**d) Static members (2 points).** All Room objects share the same maximum permitted occupancy. Explain why a class constant is suitable.

**e) Immutable strings (2 points).** Explain why calling `name.toUpperCase()` alone does not change the string stored in `name`. Show how to retain the result.

## THEORY · 60 MINUTES IN TOTAL

## Task 2: Object-Oriented Reasoning

15 points

**a) Inheritance and super (5 points).** Given the superclass below, write a subclass `Bicycle` with a constructor `Bicycle(String id)` that initializes the inherited state. Override `label()` to return `Bike:` followed by the superclass label. Keep the field private and use `super` in both places.

```
1  class Vehicle {
2      private String id;
3      public Vehicle(String id) { this.id = id; }
4      public String label() { return id; }
5  }
```

Java

**b) Overloading and overriding (5 points).** Read this separate program. State its two output lines (2 points). Identify the overloaded methods (1 point), the overridden method (1 point), and explain why the first line uses `Bell.sound()` (1 point).

```
1  class Signal {
2      public String sound() { return "beep"; }
3      public String sound(int count) { return "count=" + count; }
4  }
5  class Bell extends Signal {
6      @Override
7      public String sound() { return "ding"; }
8  }
9  // Inside main:
10 Signal signal = new Bell();
11 System.out.println(signal.sound());
12 System.out.println(signal.sound(2));
```

Java

**c) Interface or abstract class? (5 points).** Unrelated classes must all offer `double energyUse()`. Give a suitable interface declaration (2 points). A family of appliances additionally shares a private name field, a constructor, and a concrete getter. Explain why an abstract superclass can hold this shared implementation (2 points). Can that abstract superclass be instantiated directly (1 point)?

## THEORY · 60 MINUTES IN TOTAL

## Task 3: True or False

**16 points**

Mark exactly one column for each statement. Each correct answer earns **2 points**. An incorrect, missing, or double-marked answer earns 0 points; there are no deductions.

| No. | Statement                                                                                                 | True | False |
|-----|-----------------------------------------------------------------------------------------------------------|------|-------|
| 1   | A local variable of a class type stores a reference to an object, or <code>null</code> .                  |      |       |
| 2   | Declaring an array reference <code>final</code> prevents changes to its elements.                         |      |       |
| 3   | A subclass inherits its superclass's constructors.                                                        |      |       |
| 4   | A class can implement two interfaces while extending one class.                                           |      |       |
| 5   | A static method can read an instance field if it has a suitable object reference and access is permitted. |      |       |
| 6   | <code>String.equals</code> compares the text content of two strings.                                      |      |       |
| 7   | The first valid index of a nonempty Java array is 1.                                                      |      |       |
| 8   | A method declared <code>void</code> must end with <code>return null;</code> .                             |      |       |

## THEORY · 60 MINUTES IN TOTAL

## Task 4: Code Completion

12 points

Complete gaps **A-F** directly in the listing (**2 points per gap**). WaterTank stores an integer water level from **0 through 100 inclusive**. Invalid constructor or setter arguments must throw `IllegalArgumentException` without changing an existing level. The constructor must reuse the setter. `freeCapacity()` returns `100 - level`. The string format is exactly `WaterTank[level=25]` when the level is 25.

```
1  public class WaterTank {
2      // A: declare the private int field
3      -----
4
5      public WaterTank(int level) {
6          // B: initialize through the setter
7          -----
8      }
9      public int getLevel() {
10         // C
11         -----
12     }
13     public void setLevel(int level) {
14         // D: validate first, then update the field
15         -----
16         -----
17         -----
18         -----
19     }
20     public int freeCapacity() {
21         // E
22         -----
23     }
24     @Override
25     public String toString() {
26         // F
27         -----
28     }
29 }
```

Java

## THEORY · 60 MINUTES IN TOTAL

## Task 5: Code Debugging

12 points

Find **six independent defects**. For each, identify the location, explain the problem, and give a correction (**1 point for the explanation, 1 for the correction**). The specification requires all instance fields to be private; the constructor to retain its arguments; `hasName` to compare text content; and `sum` to add every array element exactly once. Assume names and arrays are non-null. An empty array must sum to 0. Missing getters, formatting choices, and Java's default field initialization are **not** defects.

```
1      public class ScoreCard {
2          public String name;
3          private int bonus;
4
5          public ScoreCard(String name, int bonus) {
6              this.name = name
7              bonus = bonus;
8          }
9          public static int getBonus() {
10             return bonus;
11         }
12         public boolean hasName(String other) {
13             return name == other;
14         }
15         public int sum(int[] scores) {
16             int total = 0;
17             for (int i = 0; i ≤ scores.length; i++) {
18                 total += scores[i];
19             }
20             return total;
21         }
22     }
```

Java

## PROGRAMMING · 30 MINUTES

## Task 6: Room Sensor Log

40 points

Build a small console application that stores room measurements and calculates a summary. Use three files: `Reading.java`, `SensorLog.java`, and `Demo.java`. You do **not** need to implement keyboard input, editing, searching, or deletion.

### Data and behavior

- A `Reading` has a room name (`String`) and an integer temperature in °C (`int`). Once created, it has no methods that change these values.
- The `Reading` constructor receives valid values; validation is the responsibility of `SensorLog.addReading`.
- A valid room name is non-null and not empty after `trim()`. Store the trimmed name. Temperatures must be between **-20 and 50 inclusive**.
- Repeated room names are allowed: they represent separate measurements.
- `addReading(String room, int temperature)` returns `true` after adding a valid reading. Otherwise it returns `false` and leaves the list unchanged.
- `countAbove(int limit)` returns the number of readings **strictly greater than** the limit.
- `averageTemperature()` returns a double average over all stored readings. It returns `0.0` when there are none. Avoid integer division.
- `displayAll()` prints each reading in insertion order, or `No readings` when empty.

### Implementation and points

| Part | Deliverable                                                                                                                          | Points |
|------|--------------------------------------------------------------------------------------------------------------------------------------|--------|
| a    | Create the named project folder and the three Java files with matching public classes.                                               | 3      |
| b    | <code>Reading</code> : two private fields, constructor, both getters, and a readable <code>toString()</code> containing both values. | 5      |
| c    | <code>SensorLog</code> : private <code>ArrayList&lt;Reading&gt;</code> initialized as an empty list.                                 | 4      |
| d    | Implement boolean <code>addReading(String room, int temperature)</code> with the specified validation.                               | 6      |
| e    | Implement <code>int countAbove(int limit)</code> ; an empty log returns 0.                                                           | 5      |
| f    | Implement <code>double averageTemperature()</code> including its empty-log behavior.                                                 | 6      |
| g    | Implement <code>void displayAll()</code> including the empty message.                                                                | 3      |
| h    | <code>Demo.main</code> : demonstrate the acceptance checks on the next page with labelled output or assertions.                      | 5      |
| i    | Use clear names, consistent indentation, and encapsulation; keep log operations in <code>SensorLog</code> , not <code>Demo</code> .  | 3      |

## Task 6: Acceptance checks and API reminder

Use a fresh log for this sequence:

1. Display it while empty; check that its average is 0.0 and countAbove(20) is 0.
2. Add (" Kitchen ", 20) and ("Kitchen", 21); both calls must return true. The displayed room names must have no surrounding spaces.
3. Check that the average is 20.5, countAbove(20) is 1, and countAbove(21) is 0.
4. Try a null room name, a spaces-only room name, and a temperature of 51; all must return false. The average and counts must remain unchanged.
5. Display the two valid readings in their insertion order.

ArrayList needs import java.util.ArrayList;. These operations may help:

```
1  ArrayList<Reading> readings = new ArrayList<>();
2  readings.add(new Reading("Hall", 18));
3  int size = readings.size();
4  boolean empty = readings.isEmpty();
5  for (Reading reading : readings) {
6      System.out.println(reading);
7  }
8  // For non-null text:
9  boolean blank = text.trim().isEmpty();
```

Java

Submit all three .java files. A compiling partial solution can earn credit for the parts it implements correctly.